Histórico de hoje desde as 14h

ISAC Brechó · 28/10/2026 · horário de Recife (UTC-3). Recorte solicitado: das 14h ao levantamento das 16h34. Estado do aplicativo conferido na main c78c923. Este relatório detalha implementação, correções, integração, arquivos e evidências, em vez de resumir a trajetória inteira do projeto.

Implementação	O que mudou e por que serve
15h30 · caa3fe6	Correção de cadastro/login locais, validações e permissões; perfil H2 persistente; ícones originais do Figma e primeiras evidências HTTP.
15h38 · ded4ef1	Painel para o brechó cadastrar e administrar peças; upload no formulário, catálogo publicado, espaçamento dos campos e testes de anúncios.
15h39 · 3975fa1	Merge do PR 8 na branch de cadastro/login/pagamento.
15h41 · f663a63	Merge do PR 7 na main, preservando o trabalho da equipe.
16h27 · e31f104	Consolidação do catálogo real, pedido manual, estoque, idempotência, divisão 10/90, ambiente e favoritos atualizados.
16h27 · 9035992	Primeiros PDFs, histórico em texto e guia de execução no VS Code.
16h27 · ddf90e4	Alinhamento da branch de integração com a main.
16h29 · c78c923	Merge do PR 9 na main oficial; workspace local alinhado à mesma versão.

Como ler os horários

São horários dos commits, não um cronômetro de cada ação. O Git não permite afirmar em que minuto cada linha foi escrita. O ponto de comparação anterior às 14h é 4b84704, registrado às 13h59:07. As implementações da equipe já existentes nessa base são indicadas como reutilizadas, não como criadas nesta tarde.

Frontend telas e navegação

A base React, TypeScript e Vite já existia. Nesta tarde, as telas passaram a usar os serviços reais de anúncios, lojas, encontros e pedidos. App.tsx concentra rotas e composição; os novos módulos foram separados em componentes e serviços.

Implementação	O que mudou e por que serve
Página inicial	A vitrine passou a usar anúncios da API com estoque disponível. Os atalhos de categorias mantêm os desenhos originais normalizados. A agenda consulta encontros do servidor, com estado vazio quando não há confirmação.
Catálogo e favoritos	Produtos da API substituem o conjunto ilustrativo como fonte do fluxo de compra. Busca, categoria, ordenação e favoritos operam sobre o catálogo carregado; não há produtos fictícios como fallback de erro.
Detalhe da peça	Usa os dados reais carregados no catálogo, foto do upload e preço do anúncio. Tamanho, cor e conservação não informados deixam de ser apresentados como fatos inventados: a tela orienta consultar a descrição.
Sacola	Mantém IDs sem duplicação e uma unidade por peça na interface. Dados locais usam chaves específicas do catálogo integrado. O total exibido é estimativa; o servidor recalcula no pedido.
Checkout	A página deixa de oferecer cartão/PIX simulados. Exige CLIENTE, recusa peças que saíram do catálogo e envia o pedido real para confirmação manual.
Perfil e pedidos	Perfil oferece atalhos para pedidos e, em conta LOJA, painel de peças. Pedidos.tsx mostra histórico do cliente ou pedidos recebidos, valores, itens e ações permitidas.
Lojas e encontros	Comunidade.tsx consulta GET /lojas e GET /encontros. Lojas têm dados, WhatsApp quando válido e ligação ao catálogo pelo vendedor. A seleção por vendedor é feita sobre os dados carregados na interface.
Estados da interface	Incluídos carregamento, erro, lista vazia, tentar novamente, paginação e bloqueio durante envio. A rota de peças publicadas redireciona ao catálogo integrado, evitando duas vitrines desconectadas.

Arquivos principais

frontend/src/App.tsx; components/Comunidade.tsx; components/Pedidos.tsx; components/Pagamento.tsx; services/anuncios.ts; services/pedidos.ts; data/store.ts. Os novos caminhos não transformam os formulários institucionais de contato e parceria em envios reais.

Frontend cadastro e painel da loja

O painel de peças foi criado nesta tarde. O formulário de cadastro por tipo de conta já existia; seus campos, validações, espaçamento e comunicação com a API foram corrigidos e conectados ao fluxo atual.

Implementação	O que mudou e por que serve
Cliente e brechó	Seletor de CLIENTE/LOJA com estado acessível. WhatsApp aparece para LOJA, é normalizado para dígitos e validado com DDI/DDD. Nome, e-mail e senha seguem o contrato do backend.
Campos e organização	Rótulos, ajuda, foco e espaçamento receberam regras comuns em forms.css. Preço e estoque usam dupla de campos responsiva; textos e controles evitam sobreposição em telas estreitas.
Acesso ao painel	Sem sessão, a tela oferece login. CLIENTE recebe indicação de área exclusiva. LOJA carrega apenas seus anúncios. O bloqueio de interface complementa, mas não substitui, a autorização do servidor.
Publicação	Formulário de título, categoria, preço, estoque disponível, descrição e foto. A categoria vem da API. Para uma peça nova, a interface exige foto e valida preço positivo e estoque inteiro não negativo.
Upload autenticado	A foto é enviada antes do anúncio, como FormData no campo arquivo. O navegador define o boundary; a aplicação não fixa Content-Type de multipart. Só então a URL devolvida é vinculada à peça.
Prévia da foto	Object URL permite visualizar o arquivo escolhido; o endereço temporário é liberado ao trocar o arquivo ou desmontar o componente. Na edição, a imagem existente é mantida quando não há nova foto.
Editar e ocultar	É possível editar dados, ocultar e reativar anúncios. Ocultar não apaga o registro. Lista e paginação são atualizadas após a operação. Anúncios VENDIDO não oferecem a ação de visibilidade.
Estoque desatualizado	A edição envia a versão recebida da API. Conflito orienta recarregar; o botão Recarregar peças e estoque permite reabrir a edição atual. O rótulo esclarece que o valor informado exclui reservas.
Envio e feedback	Requisições repetidas são bloqueadas durante envio. Há mensagens de sucesso, erro, sessão expirada, falta de permissão e arquivo inválido. O catálogo é notificado quando uma peça muda.

Arquivos

components/Cadastro.tsx, Login.tsx e ProdutosLoja.tsx; services/usuarios.ts e anuncios.ts; styles/forms.css.

Login sessão e autorização

A autenticação JWT e as senhas BCrypt já faziam parte da base anterior às 14h. O trabalho de hoje corrigiu a conexão, endureceu validações e conferiu o fluxo real de cliente e loja.

Implementação	O que mudou e por que serve
Falha do fetch	O fetch padrão deixou de ser executado como método de uma instância com receptor incompatível. A função agora delega a globalThis.fetch(...args), corrigindo falha que podia impedir a chamada HTTP antes de chegar à API.
Resposta inesperada	O frontend passou a rejeitar resposta nula, token vazio, ID inválido, usuário incompleto e papel não reconhecido. Cadastro que devolve ADMIN também é recusado no cliente.
Sessão local	obterSessao valida usuário, expiração numérica e perfil. Sessão vencida ou inválida é removida; chamadas que recebem 401 limpam o token e pedem novo login. O app consulta /auth/me ao retomar sessão.
Token nas chamadas	O serviço HTTP comum insere Authorization: Bearer nas operações privadas. Chamadas de gestão exigem LOJA na interface; pedidos usam sessão de CLIENTE ou LOJA conforme a operação.
Cadastro público	UsuarioService agora aceita publicamente apenas CLIENTE ou LOJA. Para LOJA, exige WhatsApp de 10 a 13 dígitos e remove o preenchimento fictício com zeros. A criação transacional de usuário e perfil de loja já existia e foi preservada.
Rotas privadas	/anuncios/meus e /lojas/me são protegidas antes dos caminhos públicos com curinga. A listagem de anúncios próprios passa a exigir LOJA. Criar pedido exige CLIENTE.
Propriedade do recurso	O dono do anúncio vem do usuário autenticado. Pedido só pode ser consultado/alterado pela conta envolvida. Saber o ID de outra peça ou pedido não concede permissão para editar ou confirmar.
Tratamento HTTP	401 indica ausência/expiração de sessão; 403, operação sem permissão; 404, registro não encontrado; 400, dados/regra inválidos; 409, conflito de estoque, repetição ou edição. Mensagens aparecem na interface.

Limite da sessão

O JWT permanece em localStorage e logout remove sua cópia local; não há cookie HttpOnly, refresh token ou revogação individual. Esses recursos não foram implementados nesta tarde. A validade padrão foi reduzida para oito horas.

Ícones originais 1

Todos os itens abaixo foram exportados do Figma no período e integrados ao módulo IsacIcons.tsx. A coluna de uso distingue aplicação efetiva, disponibilidade no módulo e arquivo antigo preservado. Nenhum desenho foi recriado por caminhos inventados no código.

Implementação	O que mudou e por que serve
Home Home.png	Início na navegação inferior. Cor e fundo acompanham rota ativa.
Search Search.png	Busca do catálogo e navegação Explorar.
Heart Heart.png	Arquivo inicial de favoritos. Foi exportado, mas o componente Heart agora usa a etiqueta com estrela nos dois novos SVGs.
CalendarDays CalendarDays.png	Encontros na navegação e nos atalhos de perfil.
UserRound UserRound.png	Perfil, identificação de cliente e atalhos relacionados à conta.
Shirt Shirt.svg, Shirt1.svg	Desenho de roupa em duas partes, mantido no wrapper. O atalho de categoria usa a versão normalizada categoria-roupas.svg.
Gem Gem.svg	Desenho de acessório exportado e disponibilizado no wrapper; categoria usa categoria-acessorios.svg.
Footprints Footprints.svg	Desenho de calçado exportado e disponibilizado no wrapper; categoria usa categoria-calcados.svg.

Ícones originais 2

Todos os itens abaixo foram exportados do Figma no período e integrados ao módulo IsacIcons.tsx. A coluna de uso distingue aplicação efetiva, disponibilidade no módulo e arquivo antigo preservado. Nenhum desenho foi recriado por caminhos inventados no código.

Implementação	O que mudou e por que serve
Leaf Leaf.svg	Conteúdo de propósito e sustentabilidade, preservando cores do desenho.
HandHeart HandHeart.svg	Apoio à ISAC, impacto social e informações sobre a divisão do pedido.
Store Store.svg	Identificação de brechó, área de loja e ausência de foto no painel.
ShoppingBag ShoppingBag.png	Sacola, contador de itens e ação de adicionar no detalhe.
SlidersHorizontal SlidersHorizontal.png	Abrir filtros no catálogo, com estado expandido.
Plus Plus.png	Adicionar, publicar e abrir detalhes. O estado aberto pode usar rotação; geometria do desenho não é refeita.
Trash2 Trash2.png	Remover item/dados locais nos controles que já usam esse símbolo.
ArrowRight ArrowRight.svg	Continuar, enviar e avançar; ArrowLeft deriva desse desenho por rotação de 180 graus.

Ícones originais 3

Todos os itens abaixo foram exportados do Figma no período e integrados ao módulo IsacIcons.tsx. A coluna de uso distingue aplicação efetiva, disponibilidade no módulo e arquivo antigo preservado. Nenhum desenho foi recriado por caminhos inventados no código.

Implementação	O que mudou e por que serve
ArrowUpRight ArrowUpRight.svg	Acessar/explorar em chamadas e ligações do app.
ChevronRight ChevronRight.svg	Entradas de menu e avanço em listas.
ChevronDown ChevronDown.svg	Disponibilizado para opções/recolhimento. A presença no inventário não significa uma tela nova própria.
MapPin MapPin.png	Disponibilizado para localização. Endereços e locais vêm dos dados; não foi criado mapa interativo.
Info Info.png	Avisos e explicações em telas institucionais e de uso.
CheckCircle2 CheckCircle2.svg	Conclusão, confirmação de formulário e pedido registrado, sem indicar pagamento aprovado.
Check Check.svg	Feedback de ação e avisos de interface.

Ícones alinhamento e estados

O inventário original tem 23 entradas e 24 arquivos de desenho, pois Shirt usa duas partes. Foram acrescentados quatro exports normalizados de categorias e dois SVGs de favoritos. Ao final, há 30 arquivos de arte e um manifesto de origem.

Implementação	O que mudou e por que serve
categoria-roupas.svg	Atalho de roupas em área de 24 px, com espaço óptico. Corrige escala e centralização sem deformar a camiseta.
categoria-acessorios.svg	Atalho de acessórios normalizado para o mesmo conjunto, sem usar o contorno bruto do diamante para dimensionar a célula.
categoria-calcados.svg	Calçado com margem e centro óptico consistentes. Evita o aspecto deslocado junto da barra de categorias.
categoria-favoritos.svg	Export anterior normalizado mantido no repositório; o atalho em uso passou a usar a etiqueta da nova versão. Não deve ser confundido com o arquivo atual de estado.
favoritos-nao-salvo.svg	Etiqueta original com estrela creme; estado padrão nos produtos. Origem Figma 249:557.
favoritos-salvo.svg	Mesma geometria com estrela laranja; produto salvo, rota Salvos ativa e atalho de categoria. Origem Figma 249:565.
Estados e acessibilidade	Produto mantém aria-pressed; navegação usa aria-current. Fundo, contorno e cor reforçam a seleção. Foco visível, redução de movimento e aparência de desabilitado foram tratados.
Geometria e cor	Ícones funcionais usam máscara alfa com currentColor quando apropriado; desenhos multicoloridos preservam sua arte. Favoritos usa dois arquivos completos, sem máscara que apague a estrela.
Figma e rastreabilidade	Desenho original 242:216; conjunto de variantes 249:573; quadro de revisão 249:574. manifest.json registra fonte, arquivos, estados e uso.
Exceções mantidas	X de fechar e Eye/EyeOff de visibilidade da senha continuam em Lucide por não haver substitutos definidos nesse inventário. Checkbox/radio usam controles nativos com accent-color; não foram redesenhados.

Banco de dados o que foi criado

A criação de pedidos e itens e o versionamento dos anúncios são mudanças desta tarde. As entidades de usuário, loja, categoria, imagem e encontro já existiam e foram reutilizadas; não é correto apresentar todo o banco como novo.

Implementação	O que mudou e por que serve
Pedido · tabela pedidos	Nova entidade com id, cliente, vendedor, chave de repetição, assinatura da requisição, estado, data de criação, total, valorIsac e valorBrecho. Associação com itens por cascata.
Unicidade do pedido	Restrição única combina cliente_id e chave. A mesma intenção de compra do mesmo cliente não deve gerar registros duplicados. A assinatura também detecta troca indevida de conteúdo com a mesma chave.
Valores financeiros	Total, ISAC e brechó usam BigDecimal com precisão 16 e escala 2. O servidor usa cálculo decimal e arredondamento de centavos, não valores de ponto flutuante enviados pelo navegador.
ItemPedido · itens_pedido	Nova entidade com id, pedido, anúncio, título registrado, quantidade e preço unitário. Relações obrigatórias e criação/remoção em cascata pelo pedido. Preço unitário com precisão 10 e escala 2.
Snapshot da peça	Título e preço são copiados para o item no momento da solicitação. Uma edição posterior do anúncio não reescreve os dados financeiros daquele pedido.
Anuncio · versao	Campo Long com @Version e valor inicial zero. A versão entra no request de edição e no response. Serve para detectar formulário desatualizado que poderia repor estoque reservado.
LojaResponse	Passa a devolver vendedorId, permitindo que o frontend relacione loja e peças pelo usuário vendedor. O vínculo existente da loja com Usuario é preservado.
Sem migração versionada	O perfil local usa atualização de esquema do Hibernate. Não foi criado Flyway/Liquibase nem executada migração em PostgreSQL de produção. O guia diferencia criação inicial de schema e validação posterior.

Banco transações e concorrência

Registrar uma compra exige consistência entre a linha do pedido, os itens e o estoque. Essas regras foram implementadas em PedidoService e nos métodos de bloqueio dos repositórios.

Implementação	O que mudou e por que serve
Transação de criação	O serviço valida perfil CLIENTE e chave; bloqueia o usuário comprador; reúne itens sem repetição; bloqueia anúncios em ordem de ID; valida ativo/estoque/vendedor; grava itens e total e reduz estoque. Tudo participa da mesma transação.
Uma loja por pedido	Se houver vendedores diferentes, o pedido é recusado. A interface também impede misturar brechós na sacola. Isso simplifica aceite, reserva e divisão do valor.
Última unidade	AnuncioRepository.bloquear usa PESSIMISTIC_WRITE. Duas solicitações não podem confirmar a mesma unidade disponível. O teste concorrente obteve um 201 e um 409.
Falha no meio da sacola	Se uma peça não estiver disponível ou for de outra loja, a transação reverte a reserva já feita nas peças anteriores. Não fica pedido parcial nem estoque parcialmente consumido.
Repetição da confirmação	Mesma chave e mesmos itens retornam o pedido existente. A mesma chave com outra assinatura de itens gera conflito. O bloqueio do cliente também serializa cliques simultâneos daquela conta.
Cancelar uma vez	O pedido é bloqueado antes de mudar de estado. Se já estiver CANCELADO, retorna o próprio registro. Nas demais transições permitidas, os anúncios são bloqueados e as quantidades devolvidas uma vez.
Edição de anúncio	AnuncioService usa o mesmo bloqueio de linha e compara a versão recebida com a do banco. Divergência gera 409; flush atualiza a versão devolvida. Isso evita sobrescrever a reserva com dados antigos.
Persistência local	Novo perfil local com dependência H2 em runtime apenas no perfil Maven local. Banco em arquivo sob backend/dados, modo de compatibilidade PostgreSQL e API presa ao loopback. Reinício preservou contas, pedido e estoque.

Banco já existente

Usuario continua com credenciais BCrypt e role; Loja com conta associada e contato; Categoria e Imagem com classificação/fotos; Encontro com organizador e lojas participantes. Hoje a integração dessas estruturas foi exercitada, mas suas tabelas não foram todas criadas nesta janela.

APIs conectadas autenticação e catálogo

Nesta tarde, a maioria das rotas abaixo já existia no backend. O que mudou foi a integração das telas, o cliente HTTP, a proteção das rotas privadas e a versão exigida na edição de anúncios.

Implementação	O que mudou e por que serve
POST /usuarios	Cadastro de CLIENTE/LOJA. Validações reforçadas contra ADMIN público e WhatsApp fictício. O perfil da loja é criado junto do usuário pela lógica já existente.
POST /auth/login	E-mail/senha devolvem JWT e usuário. O frontend corrigiu fetch e validou o formato da resposta antes de guardar a sessão.
GET /auth/me	Token obrigatório; usado para conferir a conta da sessão. Login errado e token inválido foram verificados.
GET /categorias	Opções reais no formulário de publicação. Categorias iniciais continuam sob o seeder existente.
GET /anuncios	Leitura pública paginada. Backend aceita título, categoria e faixa de preço; ordenação por campos permitidos e máximo de 50 itens por página. A tela carrega páginas e aplica seus filtros visuais.
GET /anuncios/{id}	Contrato de consulta existente. No fluxo atual, o detalhe da interface usa os dados carregados do catálogo; não se afirma que foi criado um endpoint novo para cada tela.
GET /anuncios/meus	Área LOJA protegida antes do curinga público; lista os anúncios da conta, inclusive ocultos. O painel usa paginação de 12.
POST /anuncios	Publicar pelo painel, com dono definido a partir do token. O navegador não envia vendedor como autoridade.
PUT /anuncios/{id}	Editar pelo proprietário, agora com versao. Edição antiga gera conflito e não substitui o estoque atual.
PATCH /anuncios/{id}/ativar ou desativar	Visibilidade da peça administrada pela loja proprietária. Ocultar é alteração de status, não exclusão física do registro.

Precisão do contrato

Não foi criado DELETE de anúncio nesta etapa. Uma permissão com esse nome no SecurityConfig não significa que exista um método de exclusão no controller. O filtro de peças por loja mostrado na interface usa vendedorId, não um parâmetro lojaId implementado hoje no endpoint.

APIs pedidos lojas agenda e fotos

Pedidos receberam controller, DTOs, service, repository e tratamento de erro novos. Lojas, encontros e uploads reaproveitam o backend da equipe e agora têm conexões de frontend identificáveis.

Implementação	O que mudou e por que serve
POST /pedidos · novo	CLIENTE envia itens e Idempotency-Key. Resposta contém ID, estado, brechó, total, valorIsac, valorBrecho, data e itens; não indica pagamento processado.
GET /pedidos · novo	Lista paginada: CLIENTE vê seus pedidos; LOJA vê os que recebeu. Nenhuma conta recebe a lista geral de todas as outras.
PATCH /pedidos/{id}/confirmar · novo	Apenas a LOJA vendedora pode aceitar um pedido pendente. Cliente ou outra loja recebem recusa de permissão.
PATCH /pedidos/{id}/cancelar · novo	Cliente cancela enquanto pendente; loja proprietária também pode cancelar depois do aceite. O serviço trata repetição sem devolver estoque duas vezes.
GET /lojas e /lojas/{id}	Catálogo público de parceiros já existente. Comunidade.tsx passa a consumir a listagem real, com descrição, contato e endereço quando preenchidos.
GET/PUT /lojas/me	Contrato privado existente para a loja. Foi protegido e testado; não foi construída nesta tarde uma nova tela completa de edição do perfil comercial.
GET /encontros e /encontros/{id}	Agenda pública existente. A nova tela mostra dados da API e estado vazio. Criar/editar/cancelar encontros continua sendo contrato de LOJA no backend; não foi criado gerenciador completo de eventos no frontend.
POST /uploads	Envio autenticado já disponível no backend, conectado ao formulário novo. O campo multipart se chama arquivo; aceita JPG/PNG/WebP de até 5 MB e recebe URL gerada pelo servidor.
GET /uploads/** e /home	Leitura pública dos arquivos e agregação de home existentes. Home foi verificada por HTTP; a tela inicial também usa as consultas de catálogo e comunidade, sem inventar novas rotas.

DTOs novos e alterados

Novos: PedidoRequest com 1 a 30 itens, IDs positivos e quantidades de 1 a 99; PedidoResponse com valores e itens. Alterados: AnuncioRequest/Response com versao e LojaResponse com vendedorId. PedidoExceptionHandler devolve regras de negócio sem expor detalhes SQL.

Checkout compra e divisão ISAC

Pagamento.tsx foi refeito. A etapa anterior tinha formulário demonstrativo de cartão/PIX e protocolo local; a atual registra pedido persistente sem apresentar pagamento como aprovado.

Implementação	O que mudou e por que serve
Antes e depois	Foram retirados entrada de cartão/CVV/validade, seleção de PIX e protocolo aleatório de simulação. A confirmação agora recebe o número real do pedido do backend.
Validação na tela	Sem login, pede entrada como cliente. Conta LOJA é orientada a usar seu painel. Itens que sumiram do catálogo ou ficaram sem estoque precisam ser removidos antes de enviar.
Confirmação repetida	Uma chave por conta e composição da sacola fica em sessionStorage. Repetir a mesma intenção após falha de rede mantém a referência; o backend decide se já existe pedido. A chave é limpa ao concluir o fluxo da sacola.
AGUARDANDO_CONFIRMACAO_MANUAL	Pedido criado e estoque reservado. O texto informa que não houve cobrança online.
CONFIRMADO	O brechó aceitou a solicitação. A tela não chama isso de pago; não existe captura de cartão ou webhook de pagamento.
CANCELADO	Estoque devolvido uma vez. Histórico mantém os valores originais, com aviso de que não há repasse devido pelo pedido cancelado.
ISAC 10% / brechó 90%	Base é a soma dos produtos. ISAC recebe total × 0,10 arredondado HALF_UP para centavos; brechó fica com o restante. O servidor grava a divisão e impede total manipulado no navegador.
Exemplo de arredondamento	R$ 89,99 resulta em R$ 9,00 para ISAC e R$ 80,99 para o brechó. No teste local do navegador: R$ 85,00, com R$ 8,50 e R$ 76,50.

O que não foi implementado

Não há gateway, débito real, transferência automática, frete calculado, estorno financeiro, emissão fiscal ou expiração automática de reserva. Pagamento, retirada/entrega e conciliação dos repasses permanecem manuais. Não devem ser apresentados à ONG como automatizados.

Ambiente execução e implantação

A execução local foi organizada para funcionar sem PostgreSQL ou Docker instalados nesta máquina. Configuração local e configuração de publicação passaram a ter responsabilidades separadas.

Implementação	O que mudou e por que serve
application.properties	Credenciais fixas de PostgreSQL e chave JWT padrão foram substituídas por DB_URL, DB_USER, DB_PASSWORD e JWT_SECRET. JWT_EXPIRATION_MS tem padrão de oito horas; porta e diretório de uploads são configuráveis.
Perfil local e chave	H2 persistente em backend/dados. INICIAR-BACKEND.ps1 gera chave aleatória de 48 bytes em arquivo ignorado quando necessário. Nenhum segredo real vai para o Git.
Compose e proxy	docker-compose.yml exige usuário, senha e chave via ambiente; mantém volumes de banco/fotos e portas em loopback. deploy/nginx.conf exemplifica /api com remoção do prefixo e fallback de SPA. Docker não foi executado aqui.
Preparar o VS Code	PREPARAR-AMBIENTE.ps1 verifica ferramentas e Node 22.18+, mostra Java e instala dependências ausentes. Não reinstala um node_modules existente sem opção explícita.
Iniciar serviços	Iniciadores .cmd chamam scripts PowerShell 7. O frontend instala dependências quando ausentes e usa porta fixa 4186; backend usa 8080. Erros de instalação interrompem o fluxo em vez de seguir silenciosamente.
Workspace e testes	ISAC-Brecho.code-workspace contém preparação, frontend, backend, execução conjunta, testes dos dois lados e build. Pastas geradas e chave local ficam fora da navegação comum.
Guias e histórico	README ganhou links; GUIA-VSCODE.md traz execução e problemas comuns; INTEGRACAO-LOCAL.md explica contratos; IMPLANTACAO.md orienta ambiente, PostgreSQL, volumes e limites. PDFs acompanham a apresentação.
ZIP e arquivos privados	Pacote atualizado com fonte e documentos, sem node_modules, banco, uploads de teste, tokens, chaves e logs. Um ZIP extraído em outro local começa sem as contas e peças da demonstração.

Implantação não foi comprovada

O banco executado e reiniciado foi H2. A entrega prepara configuração para PostgreSQL, mas não registra execução de PostgreSQL, restauração de backup, Docker ou serviço público. DDL_AUTO=update é orientação de inicialização controlada, não um sistema de migrações versionadas.

Testes e evidências do período

Foram mantidos os testes existentes, ajustados os contratos afetados e acrescentada cobertura para os serviços HTTP do frontend e o fluxo transacional de pedidos. Resultado registrado: 63 testes backend, 48 frontend e build TypeScript/Vite aprovados.

Implementação	O que mudou e por que serve
Auth frontend	Novos casos de fetch com receptor correto e normalização do WhatsApp. Testes de resposta nula, usuário incompleto, ID inválido e papel inesperado foram alinhados ao AuthResponse.
Anúncios frontend · 5 casos	Token na publicação sem dono no body; CLIENTE sem chamada de gestão; multipart sem Content-Type fixo; arquivo inválido recusado; imagem local pelo proxy e erro de permissão.
Pedidos frontend · 5 casos	Body apenas com itens/quantidades e chave; sessão expirada bloqueada antes da rede; 401 limpa sessão; conflito de estoque apresenta regra; cancelamento autenticado sem preço/dono no body.
Pedidos backend · 7 cenários	Preço/divisão/persistência/idempotência/cancelamento; papel e propriedade; rollback/mistura de lojas; disputa da última unidade; preço do navegador ignorado; edição antiga recusada; repetição simultânea com uma reserva.
Testes existentes preservados	AuthController, AnuncioController, fluxo principal, serviços de usuário/anúncio/upload e inicialização do backend; CatalogoService, SacolaService, UsuarioService e contratos do frontend. Mocks de edição passaram a usar o bloqueio de linha.
HTTP local · 12 verificações	Cadastro cliente/loja, duplicidade, senha errada, login, token, acesso privado, CLIENTE impedido de criar anúncio, perfil da loja, ADMIN público recusado e WhatsApp obrigatório. Registro: validacao-integracao.json.
Produto publicado	Foto servida com 200; edição de preço; outra loja recusada com 403 e lista própria vazia; ocultar/reativar funcionaram. Registro: validacao-produtos.json.
Navegador e reinício	Cliente solicitou pedido, loja aceitou e cancelou, estoque voltou. Pedido e sessão preservados após reiniciar a API. Ícones conferidos salvo/não salvo e preparação do ambiente executada. Registro: validacao-mvp.json e revisão visual.

Interpretação dos resultados

Os 111 testes não significam 111 testes novos: são as duas suítes completas aprovadas. A cobertura de banco usa H2 e não comprova comportamento em PostgreSQL de produção. Nenhum pagamento real foi realizado.

Arquivos alterados 1

Inventário exato entre a base das 13h59 (4b84704) e a main consolidada às 16h29 (c78c923). A = arquivo adicionado; M = arquivo modificado. Os vetores e bitmaps aparecem nominalmente nas páginas de ícones. A tabela não trata arquivo antigo preservado como funcionalidade nova.

Tipo	Caminho no repositório
M	.gitignore
A	GUIA-VSCODE.md
A	IMPLANTACAO.md
A	INICIAR-BACKEND.cmd
A	INICIAR-BACKEND.ps1
A	INICIAR-FRONTEND.cmd
A	INICIAR-FRONTEND.ps1
A	INTEGRACAO-LOCAL.md
A	ISAC-Brecho.code-workspace
A	PREPARAR-AMBIENTE.ps1
M	README.md
A	backend/.env.example
M	backend/docker-compose.yml
M	backend/docs/AUTH-API.md
M	backend/pom.xml
M	backend/src/main/java/com/isac/bazar_solidario/config/SecurityConfig.java
A	backend/src/main/java/com/isac/bazar_solidario/controller/PedidoController.java
M	backend/src/main/java/com/isac/bazar_solidario/dto/AnuncioRequest.java

Arquivos alterados 2

Inventário exato entre a base das 13h59 (4b84704) e a main consolidada às 16h29 (c78c923). A = arquivo adicionado; M = arquivo modificado. Os vetores e bitmaps aparecem nominalmente nas páginas de ícones. A tabela não trata arquivo antigo preservado como funcionalidade nova.

Tipo	Caminho no repositório
M	backend/src/main/java/com/isac/bazar_solidario/dto/AnuncioResponse.java
M	backend/src/main/java/com/isac/bazar_solidario/dto/LojaResponse.java
A	backend/src/main/java/com/isac/bazar_solidario/dto/PedidoRequest.java
A	backend/src/main/java/com/isac/bazar_solidario/dto/PedidoResponse.java
A	backend/src/main/java/com/isac/bazar_solidario/exception/PedidoExceptionHandler.java
M	backend/src/main/java/com/isac/bazar_solidario/model/Anuncio.java
A	backend/src/main/java/com/isac/bazar_solidario/model/ItemPedido.java
A	backend/src/main/java/com/isac/bazar_solidario/model/Pedido.java
M	backend/src/main/java/com/isac/bazar_solidario/repository/AnuncioRepository.java
A	backend/src/main/java/com/isac/bazar_solidario/repository/PedidoRepository.java
M	backend/src/main/java/com/isac/bazar_solidario/service/AnuncioService.java
A	backend/src/main/java/com/isac/bazar_solidario/service/PedidoService.java
M	backend/src/main/java/com/isac/bazar_solidario/service/UsuarioService.java
A	backend/src/main/resources/application-local.properties
M	backend/src/main/resources/application.properties
A	backend/src/test/java/com/isac/bazar_solidario/controller/PedidoIntegrationTest.java
M	backend/src/test/java/com/isac/bazar_solidario/service/AnuncioServiceTest.java
A	deploy/nginx.conf

Arquivos alterados 3

Inventário exato entre a base das 13h59 (4b84704) e a main consolidada às 16h29 (c78c923). A = arquivo adicionado; M = arquivo modificado. Os vetores e bitmaps aparecem nominalmente nas páginas de ícones. A tabela não trata arquivo antigo preservado como funcionalidade nova.

Tipo	Caminho no repositório
A	docs/HISTORICO-ATUALIZACOES.md
A	docs/apresentacao/ISAC-Brecho-Guia-VS-Code.pdf
A	docs/apresentacao/ISAC-Brecho-Historico-de-Atualizacoes.pdf
A	docs/apresentacao/favoritos-figma.png
A	docs/figma-favoritos.json
A	docs/historico-conteudo.json
M	frontend/README.md
M	frontend/src/App.tsx
M	frontend/src/components/Cadastro.tsx
A	frontend/src/components/Comunidade.tsx
A	frontend/src/components/IsacIcons.tsx
M	frontend/src/components/Login.tsx
M	frontend/src/components/Pagamento.tsx
A	frontend/src/components/Pedidos.tsx
A	frontend/src/components/ProdutosLoja.tsx
M	frontend/src/data/store.ts
M	frontend/src/main.tsx
A	frontend/src/services/anuncios.ts

Arquivos alterados 4

Inventário exato entre a base das 13h59 (4b84704) e a main consolidada às 16h29 (c78c923). A = arquivo adicionado; M = arquivo modificado. Os vetores e bitmaps aparecem nominalmente nas páginas de ícones. A tabela não trata arquivo antigo preservado como funcionalidade nova.

Tipo	Caminho no repositório
A	frontend/src/services/pedidos.ts
M	frontend/src/services/usuarios.ts
A	frontend/src/styles/forms.css
A	frontend/src/styles/icons.css
M	frontend/tests/UsuarioService.test.ts
A	frontend/tests/anuncios.test.ts
A	frontend/tests/integracao-auth.test.ts
A	frontend/tests/pedidos.test.ts
M	frontend/tests/usuarios.test.ts
M	frontend/tsconfig.json
A	tools/gerar-documentos.py
A	validacao-integracao.json
A	validacao-mvp.json
A	validacao-produtos.json

Conferência do recorte e referências

Este documento substitui o resumo amplo por um registro do trabalho desta tarde. Não inclui como novidade de hoje a criação da logo, a paleta original, a estrutura inicial do aplicativo nem todas as tabelas e APIs que a equipe já havia desenvolvido.

O que foi confirmado

Base 4b84704 às 13h59:07; alterações consolidadas em c78c923 às 16h29:15; levantamento às 16h34. Repositório oficial: Residencia-Tecnologica-2026-2/bazar-solidario. PRs 8, 7 e 9 registram as integrações do período.

O que foi entregue nesta tarde

Conexão e autorização corrigidas; todos os exports de ícones listados; ajustes de escala e estados; painel de anúncios; catálogo/lojas/agenda conectados; pedido persistido com estoque e divisão ISAC; configuração local e de ambiente; testes, evidências e documentos de execução.

O que não deve ser afirmado

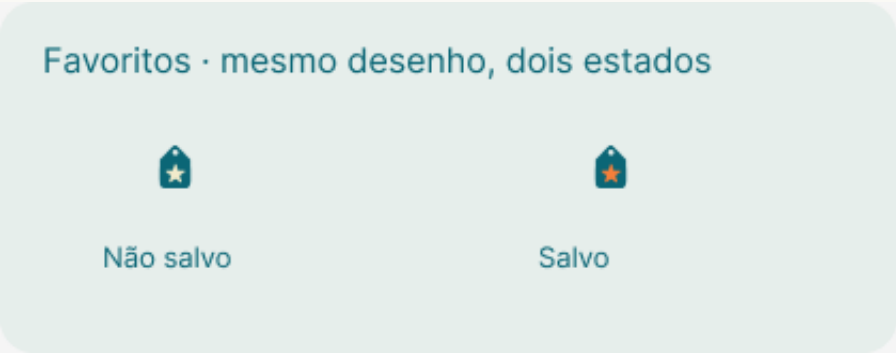
Não há comprovação de PostgreSQL/Docker em execução nesta máquina, nem implantação pública, gateway de pagamento, repasse automático, expiração automática da reserva ou novas telas completas de gestão de eventos e perfil comercial.

Fontes verificáveis

Git diff 4b84704..c78c923; manifest.json dos ícones; entidades, controllers e services atuais; validacao-integracao.json, validacao-produtos.json e validacao-mvp.json; logs locais de testes; variantes de favoritos 249:557 e 249:565 no Figma.

Como apresentar à ONG

Comece pelo problema resolvido, mostre a interface e o painel de publicação, registre um pedido como cliente e confira o aceite pela loja. Explique a reserva e o cálculo de 10% para ISAC. Use as páginas técnicas como apoio a perguntas, sem chamar confirmação manual de pagamento.



Repositório oficial no GitHub · Favoritos no Figma